

# МЕТОДИЧЕСКИЕ УКАЗАНИЯ

## УП.03. Учебная практика

### Этап 5. Безопасность и эксплуатационные риски

#### Использование методов защиты ПО компьютерных систем

| Параметр           | Значение                                                                                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Практика           | УП.03. Сопровождение и обслуживание программного обеспечения компьютерных систем                                                                            |
| Этап 5             | Безопасность и эксплуатационные риски программной системы                                                                                                   |
| Связь с этапом 3   | Проект уже развернут как сервис, demo-стенд, PaaS-развертывание или готовый билд                                                                            |
| Связь с этапом 4   | Качество было проверено инструментами: тесты, DevTools, Lighthouse, Postman/curl, pytest, логи                                                              |
| Основной результат | Проведена базовая проверка безопасности и устойчивости: секреты, .gitignore, роли, доступ к чужим данным, CORS, backup/restore, зависимости, открытые порты |
| Формат сдачи       | Ссылка на репозиторий + SECURITY_REPORT.docx + SECURITY_CHECKLIST.md + RISK_REGISTER.md + BACKUP_RESTORE_REPORT.md + скриншоты                              |
| Инструменты        | OWASP Top 10 checklist, GitHub secret scan вручную, dependency check, npm audit/pip-audit, backup tools, firewall basics, docker logs, netstat/ss           |

#### Главная мысль этапа:

Студент должен не “написать, что проект защищен”, а проверить конкретные слабые места: не утекли ли секреты, нельзя ли посмотреть чужие данные, есть ли резервная копия, восстанавливается ли БД, нет ли устаревших зависимостей и лишних открытых портов.

## Что нужно сдать

Сдаётся не большой реферат по информационной безопасности, а короткий комплект доказательств: что именно проверено, какие проблемы найдены, что исправлено и что осталось риском.

#### Структура папки сдачи

```

УП03_Этап5_Иванов_Иван_Группа/
|-- repo_link.txt
|-- docs/
|   |-- SECURITY_CHECKLIST.md
|   |-- RISK_REGISTER.md
|   |-- BACKUP_RESTORE_REPORT.md
|   |-- SECURITY_REPORT.docx
|   `-- RECOMMENDATIONS.md
-- screenshots/
|   |-- 01_gitignore_no_env.png
|   |-- 02_env_example_without_secrets.png
|   |-- 03_secret_scan_result.png
|   |-- 04_dependency_check_result.png
|   |-- 05_roles_access_check.png
|   |-- 06_foreign_data_access_denied.png
|   |-- 07_cors_or_security_config.png
|   |-- 08_backup_created.png
|   |-- 09_restore_success.png
|   |-- 10_open_ports_check.png
|   |-- 11_logs_no_critical_errors.png
|   `-- 12_security_commit.png
-- scripts/
|   |-- security-check.bat
|   |-- deps-check.bat
|   |-- backup.bat
|   |-- restore.bat
|   |-- ports-check.bat
|   `-- logs-security.bat
`-- project_files_in_repository/
    |-- .gitignore
    |-- .env.example
    -- SECURITY.md

```

```
| -- docker-compose.yml / docker-compose.prod.yml
|-- scripts/...
```

## Порядок выполнения

| Шаг | Что сделать                                                                                                  | Что приложить                                             |
|-----|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 1   | Проверить .gitignore: в репозиторий не должны попадать .env, ключи, токены, логи, дампы БД, временные файлы. | Скриншот .gitignore и списка файлов репозитория.          |
| 2   | Проверить .env.example: там должны быть только примерные значения без реальных паролей и токенов.            | Скриншот .env.example.                                    |
| 3   | Выполнить ручной поиск секретов по проекту: password, token, secret, api_key, jwt, smtp, db.                 | Скриншот результата secret scan.                          |
| 4   | Проверить устаревшие или уязвимые зависимости инструментом под свой стек.                                    | Скриншот npm audit / pip-audit / dotnet / Maven / Gradle. |
| 5   | Проверить роли и доступ: пользователь не должен видеть данные другого пользователя или admin-раздел.         | Скриншоты успешного доступа и запрета доступа.            |
| 6   | Проверить CORS, allowed hosts, debug=false, отсутствие подробных ошибок на production/demo.                  | Скриншот конфигурации или результата проверки.            |
| 7   | Создать резервную копию данных или файлов проекта.                                                           | Скриншот backup-файла или команды backup.                 |
| 8   | Проверить восстановление из резервной копии.                                                                 | Скриншот restore и проверки данных.                       |
| 9   | Проверить открытые порты, работающие сервисы и логи.                                                         | Скриншот docker ps/netstat/ss и логов.                    |
| 10  | Заполнить SECURITY_CHECKLIST.md, RISK_REGISTER.md и короткий SECURITY_REPORT.docx.                           | Документы + commit/push.                                  |

## Обязательные проверки безопасности

| Проверка      | Минимум, который нужно сделать                                                         | Доказательство                                                           |
|---------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Секреты       | Убедиться, что .env, токены, пароли, ключи и дампы БД не загружены в Git.              | .gitignore, .env.example, результат поиска секретов.                     |
| Зависимости   | Проверить зависимости на устаревшие версии и известные проблемы.                       | npm audit / pip-audit / dotnet list package --vulnerable / Maven/Gradle. |
| Роли и доступ | Проверить минимум 2 сценария доступа: разрешенный и запрещенный.                       | Скриншоты/API-ответы 200/401/403.                                        |
| Чужие данные  | Попробовать открыть ресурс другого пользователя: заявка, профиль, заказ, запись, файл. | Доступ запрещен или данные не раскрыты.                                  |
| CORS/hosts    | Для web/API проверить, что нет бездумного доступа "*" в production/demo.               | Скриншот конфига или заголовков.                                         |
| Backup        | Создать резервную копию БД/файлов.                                                     | Файл backup или команда с результатом.                                   |
| Restore       | Восстановить данные из backup и проверить, что они открываются.                        | Скриншот restore + данные после восстановления.                          |
| Порты         | Показать, какие порты открыты и какие сервисы работают.                                | docker ps / netstat / ss.                                                |
| Логг          | Проверить, что критические ошибки не повторяются после запуска.                        | Логг без постоянных traceback/fatal/error.                               |

## Скрипты этапа 5

Скрипты нужны, чтобы проверка не зависела от памяти студента. Если стек проекта другой, команды можно заменить, но названия файлов и смысл должны сохраниться.

### Пример security-check.bat

```
# scripts/security-check.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo === Проверка подозрительных слов в проекте ===
git grep -n -i -E "password|secret|token|api_key|apikey|jwt|smtp|database_url" -- . ":!docs" ":!screenshots" ":!*md"

echo === Проверка статуса Git ===
git status --short

echo Если найдены реальные пароли/токены — удалить их из проекта и заменить на .env.example.
```

**Пример deps-check.bat**

```
# scripts/deps-check.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo Выберите команду под свой стек и оставьте только нужную:
REM Python:
REM pip-audit
REM pip list --outdated

REM Node.js:
REM npm audit
REM npm outdated

REM .NET:
REM dotnet list package --vulnerable

REM Java/Maven:
REM mvn org.owasp:dependency-check-maven:check
```

**Пример ports-check.bat**

```
# scripts/ports-check.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo === Docker-контейнеры и порты ===
docker compose ps

echo === Порты Windows ===
netstat -ano | findstr LISTENING
```

**Backup и restore**

Если в проекте есть база данных, проверка backup/restore обязательна. Если базы нет, студент делает backup важных файлов: конфиги, данные, uploads, sqlite-файл, json/csv-хранилище.

| Тип проекта         | Что сделать                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL в Docker | Сделать pg_dump из контейнера, сохранить файл backup.sql, затем восстановить в тестовую БД или пересозданный контейнер. |
| SQLite              | Скопировать .db-файл в папку backups, удалить/переименовать исходный файл, восстановить копию и запустить проект.       |
| Файловое хранилище  | Скопировать папку uploads/data, затем восстановить и проверить, что файлы открываются.                                  |
| Проект без данных   | Написать в отчёте: “постоянное хранилище не используется”, но сделать backup конфигурации и release-архива.             |

**Пример backup.bat**

```
# scripts/backup.bat для PostgreSQL в docker compose
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

if not exist backups mkdir backups
for /f "tokens=1-4 delims=/: " %a in ("%date% %time%") do set TS=%c%%b%%a_%d

docker compose exec -T db pg_dump -U app_user app_db > backups\backup_%TS%.sql

echo Backup создан в папке backups
```

**Пример restore.bat**

```
# scripts/restore.bat для PostgreSQL в docker compose
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo Укажите имя файла backup вручную в команде ниже:
REM docker compose exec -T db psql -U app_user app_db < backups\backup_YYYYMMDD.sql

echo После восстановления откройте приложение и проверьте данные.
```

## SECURITY\_CHECKLIST.md

## Шаблон SECURITY\_CHECKLIST.md

# SECURITY\_CHECKLIST.md

```

| Проверка | Статус | Доказательство | Что исправлено |
|---|---|---|---|
| .env не загружен в Git | выполнено / не выполнено | скриншот 01 | добавлен в .gitignore |
| .env.example без реальных секретов | выполнено / не выполнено | скриншот 02 | заменены значения на change_me |
| Ручной поиск секретов выполнен | выполнено / не выполнено | скриншот 03 | удалены токены |
| Зависимости проверены | выполнено / не выполнено | скриншот 04 | обновлена зависимость |
| Проверены роли | выполнено / не выполнено | скриншот 05 | исправлен доступ |
| Проверен доступ к чужим данным | выполнено / не выполнено | скриншот 06 | добавлена проверка владельца |
| CORS/hosts/debug проверены | выполнено / не выполнено | скриншот 07 | debug=false |
| Backup создан | выполнено / не выполнено | скриншот 08 | настроен backup.bat |
| Restore проверен | выполнено / не выполнено | скриншот 09 | проверено восстановление |
| Открытые порты проверены | выполнено / не выполнено | скриншот 10 | закрыт лишний порт |

```

## RISK\_REGISTER.md

Реестр рисков должен быть коротким, но конкретным. Минимум 6 рисков: 3 по безопасности и 3 по эксплуатации.

| Риск                             | Причина                           | Вероятность    | Влияние | Мера снижения                                     |
|----------------------------------|-----------------------------------|----------------|---------|---------------------------------------------------|
| Утечка .env                      | Файл случайно попал в Git         | Средняя        | Высокое | .gitignore, .env.example, проверка git status.    |
| Доступ к чужим данным            | Нет проверки владельца ресурса    | Средняя        | Высокое | Проверять user_id/role перед выдачей данных.      |
| Уязвимая зависимость             | Старая библиотека                 | Средняя        | Среднее | npm audit / pip-audit, обновление версии.         |
| Потеря данных                    | Нет резервной копии               | Низкая/средняя | Высокое | backup.bat, restore.bat, проверка восстановления. |
| Сервис не поднимается после сбоя | Нет проверки restart/logs         | Средняя        | Среднее | restart policy, check_deploy.bat, логи.           |
| Открыт лишний порт               | Неверная настройка Docker/сервера | Низкая         | Среднее | Проверка ports, закрыть ненужные ports.           |

## Короткий SECURITY\_REPORT.docx

| Раздел отчёта                       | Что написать                                                                                                |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1. Что проверялось                  | Секреты, зависимости, роли, доступ к данным, CORS/hosts, backup/restore, порты, логи.                       |
| 2. Какие инструменты использовались | Команды и инструменты: git grep, npm audit/pip-audit, docker ps, netstat/ss, backup scripts.                |
| 3. Что было найдено                 | 1-5 конкретных проблем или указание, что критических проблем не найдено.                                    |
| 4. Что исправлено                   | Что изменено в .gitignore, .env.example, ролях, зависимостях, backup-скриптах.                              |
| 5. Какие риски остались             | Что пока не исправлено и почему: нет HTTPS, нет полноценного мониторинга, нет автоматического backup и т.д. |
| 6. Рекомендации                     | Что нужно сделать дальше: CI security check, регулярные backups, HTTPS, мониторинг, аудит прав.             |

## 9. Обязательные скриншоты

| Файл                               | Что должно быть видно                                                                             |
|------------------------------------|---------------------------------------------------------------------------------------------------|
| 01_gitignore_no_env.png            | .env, backups, logs, dumps, node_modules/.env добавлены в .gitignore.                             |
| 02_env_example_without_secrets.png | .env.example содержит только примерные значения: change_me, example, localhost.                   |
| 03_secret_scan_result.png          | Результат поиска секретов. Если найдены совпадения — видно, что это примеры, а не реальные ключи. |
| 04_dependency_check_result.png     | Результат проверки зависимостей выбранным инструментом.                                           |
| 05_roles_access_check.png          | Проверка роли/авторизации: обычный пользователь не имеет admin-доступа.                           |
| 06_foreign_data_access_denied.png  | Попытка открыть чужую запись/файл/ресурс запрещена или возвращает 403/404.                        |
| 07_cors_or_security_config.png     | CORS/allowed hosts/debug/production config.                                                       |
| 08_backup_created.png              | Создан backup-файл.                                                                               |
| 09_restore_success.png             | Данные восстановлены и проект после восстановления работает.                                      |
| 10_open_ports_check.png            | Список открытых портов/контейнеров.                                                               |
| 11_logs_no_critical_errors.png     | Логи без повторяющихся критических ошибок.                                                        |
| 12_security_commit.png             | Commit/push с файлами этапа 5.                                                                    |

## Критерии приемки этапа

| Критерий       | Принимается                                                                                | Не принимается                                                  |
|----------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Секреты        | .env не в Git, .env.example без реальных данных, выполнен поиск секретов.                  | В репозитории лежат реальные пароли, токены, дампы БД или .env. |
| Зависимости    | Есть результат проверки зависимостей и вывод по найденным проблемам.                       | Студент только написал “зависимости нормальные”.                |
| Доступ         | Проверены роли или доступ к данным, есть доказательства разрешенного/запрещенного доступа. | Нет проверки чужих данных или ролей.                            |
| Backup/restore | Создан backup и проверено восстановление.                                                  | Есть только backup без проверки restore.                        |
| Порты и логи   | Показаны работающие сервисы, порты и логи после запуска.                                   | Нет доказательств, что сервисы работают стабильно.              |
| Риски          | Есть минимум 6 конкретных рисков и меры снижения.                                          | Риски общие: “могут быть ошибки”, “может сломаться”.            |
| Отчёт          | Короткий, с конкретными выводами и ссылками на скриншоты.                                  | Большой текст без доказательств.                                |

### Итог этапа:

После этапа 5 проект должен быть не просто запущен и протестирован, а проверен на базовые эксплуатационные риски: секреты, доступ, зависимости, резервное копирование, восстановление, порты и логи. Если проблема найдена — это нормально. Главное: она должна быть зафиксирована, исправлена или внесена в реестр рисков.